

Edge cases (fix a flaky test) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper property checks, including answers with more than one correct wording). Read Basic first even if you already know the pattern — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

Start with a broken test like this: `assert response == "Your order has shipped."` Run it a few times against a real LLM call — sometimes it passes, sometimes it fails, even though the model isn't getting anything wrong. It's just wording the same correct answer differently each time.

The fix isn't to make the check looser in a vague way — it's to figure out exactly *what* made that answer correct, and check for that specific thing instead of the exact words.

Ask yourself: what's the one fact that HAD to be true for that answer to count as correct? Write that down first.

Intermediate Version

A test that sometimes passes and sometimes fails on the *same behavior* is worse than a test that always fails — a failing test gets investigated; a flaky one gets a shrug and a re-run, and is eventually ignored, while still looking like part of your safety net.

The fix is always the same shape: replace an exact-match check with a **property** check — something true of every correct answer, whatever the wording. Common properties:

- **Keyword check:** `"shipped"` in `response.lower()` — `.lower()` matters, because capital letters don't make an answer more or less correct.
- **Length limit:** `len(response.split()) < 60` — when a rule really is "keep it short".
- **One of several phrasings:** a list like `["shipped", "on its way", "dispatched"]`, and a `for` loop that sets `found = True` when any of them appears — for facts with more than one correct wording.

Watch out: a single keyword can create *new* flakiness. If the model sometimes correctly says "your order is on its way", a check for only `"shipped"` fails on a right answer.

The model also needs the facts to answer at all: put them in the prompt (for example, "Order A123 shipped today"), the same way a real support bot gets them from a tool or a database.

Difference between Basic and Intermediate: Basic finds the one fact that makes the answer correct and checks for it. Intermediate explains why flaky tests are so harmful, lists the common property checks, and handles facts with several correct wordings so the fix doesn't create a new kind of flakiness.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
broken test:
    assert response == "Your order has shipped."

fixed test:
    response = call the model
    assert "shipped" is somewhere in response, ignoring capitals
```

Here's almost the whole thing — `call_model` and `SHIPPING_PROMPT` are defined at the top of the same file (the Solution shows them):

```
# practice/llm_output_testing_practice.py – Edge cases section
def test_shipping_reply():
    response = call_model(SHIPPING_PROMPT)
    assert "shipped" in response.lower()
```

Add one more assert for a sensible length limit yourself.

Intermediate Version

```
ACCEPTABLE_PHRASES = ["shipped", "on its way", "sent out", "dispatched"]

test_shipping_reply():
    response = call the model, lower-cased
    found = False
    for each phrase: if it is in the response, found = True
    assert found (and show the response if not)
    assert the reply is under 60 words
```

Here's most of it — write the loop yourself:

```
# practice/llm_output_testing_practice.py – Edge cases section
ACCEPTABLE_PHRASES = ["shipped", "on its way", "sent out", "dispatched"]
```

```
def test_shipping_reply():
    response = call_model(SHIPPING_PROMPT).lower()
    found = False
    # your turn: loop over ACCEPTABLE_PHRASES, set found = True
    # when a phrase is in response
    ...
    assert found, response
    assert len(response.split()) < 60
```

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)